Rasters and Difference-in-Differences

ECON 895 Spatial Techniques in Empirical Economics

Noel D. Johnson
George Mason University

September 21, 2026

Plan for tonight

The potato

Raster grammar

From surface to column

The design

What it cost

We break around 8:45.

Three weeks of habits

- Count rows on both sides, and then look at what you kept.
- `st_set_crs()` relabels, `st_transform()` moves.
- Equal-area before `st_area()`. Units are a free check that catches only some of the errors.
- Move the line and the effect should die. Rebuild the published variable and compare.

Tonight adds one more, and it is the hardest to hold. A number extracted from a surface is a **measurement you designed**.

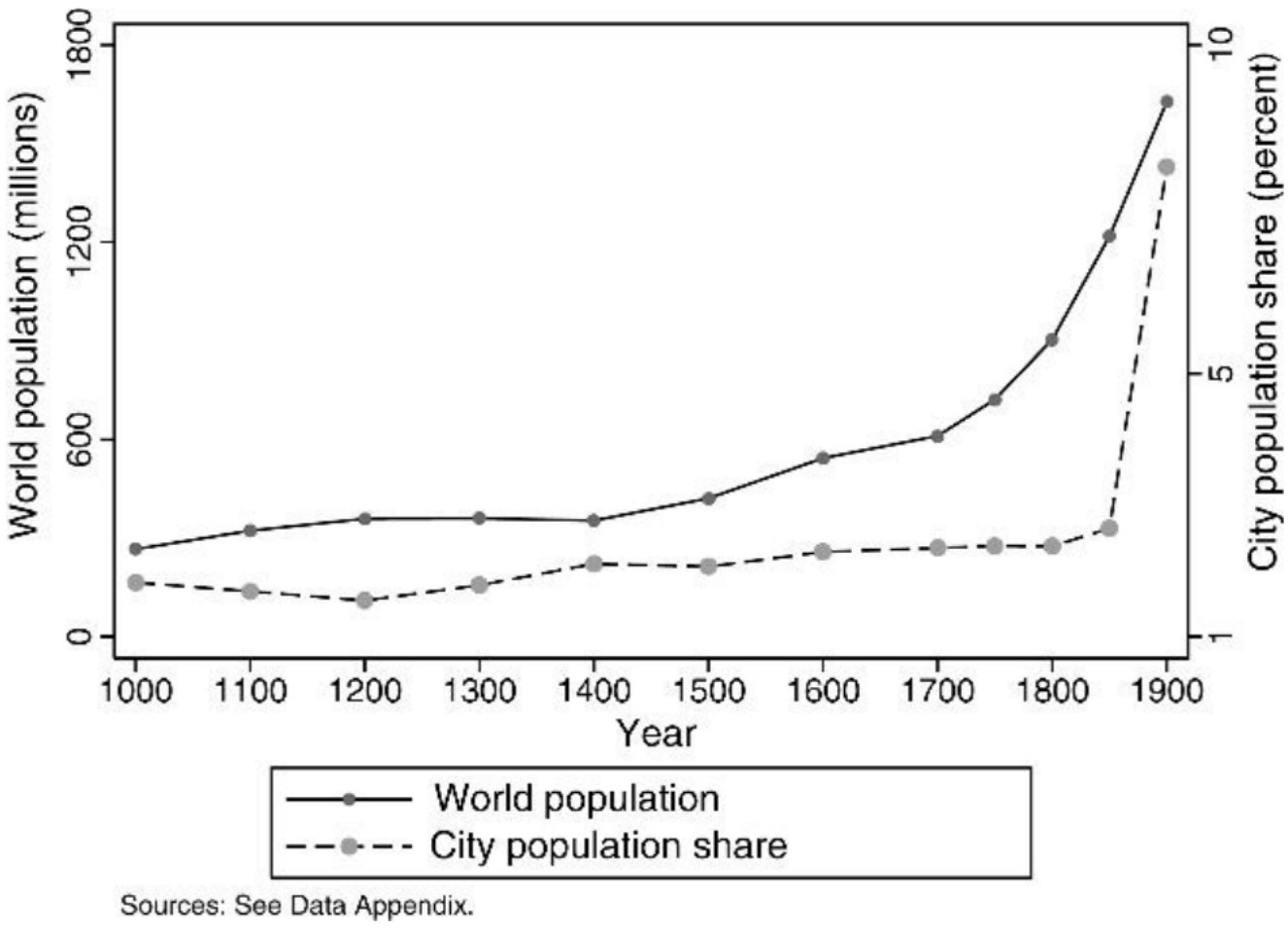
The potato

What the potato did

- The potato reaches the Old World from the Americas in the sixteenth century and spreads as a staple through the eighteenth.
- It yields more calories per acre than the grains it displaced, on land that grain does not like.
- Nunn and Qian (2011) estimate that its introduction accounts for **about a quarter** of the growth in Old World population and urbanization between 1700 and 1900.

That figure is quoted from their abstract, along with the handful of their other published numbers that appear later tonight. Everything else on these slides was computed at compile time.

World population, 1000–1900



Nunn and Qian (2011) Figure I. Population in millions on the left axis, the share living in cities on the right. The bend is in the eighteenth century, which is the fact the paper sets out to account for.

Why the obvious comparison fails

- Places suited to potatoes are cool, wet, and often upland. Unsuitable places are warm, dry, or lowland.
- Those places differed in soil, in disease, in trade access, and in politics, for reasons that have nothing to do with tubers.
- Comparing potato-suitable places with unsuitable ones measures all of it at once.

Dose and timing

Nobody's suitability ever changes

What changes is that before the potato arrives, suitability for growing it is irrelevant, and afterwards it decides the harvest. The treatment is the interaction of a fixed dose with a moment in time.

- That is a difference in differences with a continuous treatment. City fixed effects absorb every level difference. Year effects absorb everything common.

- What identifies the estimate is whether high-suitability and low-suitability places were on **the same trajectory** beforehand.

The treatment is a surface

Weeks 2 and 3 measured distance to points and lines. Suitability is neither.

A value everywhere

Defined continuously over the map, with no boundaries and no features. Every tool you own expects features.

- The other half of spatial data is the surface, and one verb turns it into a column in your regression. That is tonight.

Raster grammar

A raster stores its geometry once

```
potato <- rast("../data/gaez_potato_suitability.tif")
potato
```

```
## class       : SpatRaster
## size        : 2160, 4320, 1  (nrow, ncol, nlyr)
## resolution  : 0.08333333, 0.08333333  (x, y)
## extent      : -180, 180, -90, 90  (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (EPSG:4326)
## source      : gaez_potato_suitability.tif
## color table : 1
## name        : gaez_potato_suitability
```

A raster is a grid, an extent, a CRS, and a block of values. Week 3's 299 districts each dragged thousands of coordinates behind them. This holds 9,331,200 values and stores the geometry **once**, as four numbers and a resolution. We use `terra`. The 2022 version of this course used `raster` and `sp`, which are deprecated. The toolchain moved, exactly as `sp` moved to `sf`, and inherited code rots quietly.

run it

Their map of the file on your disk

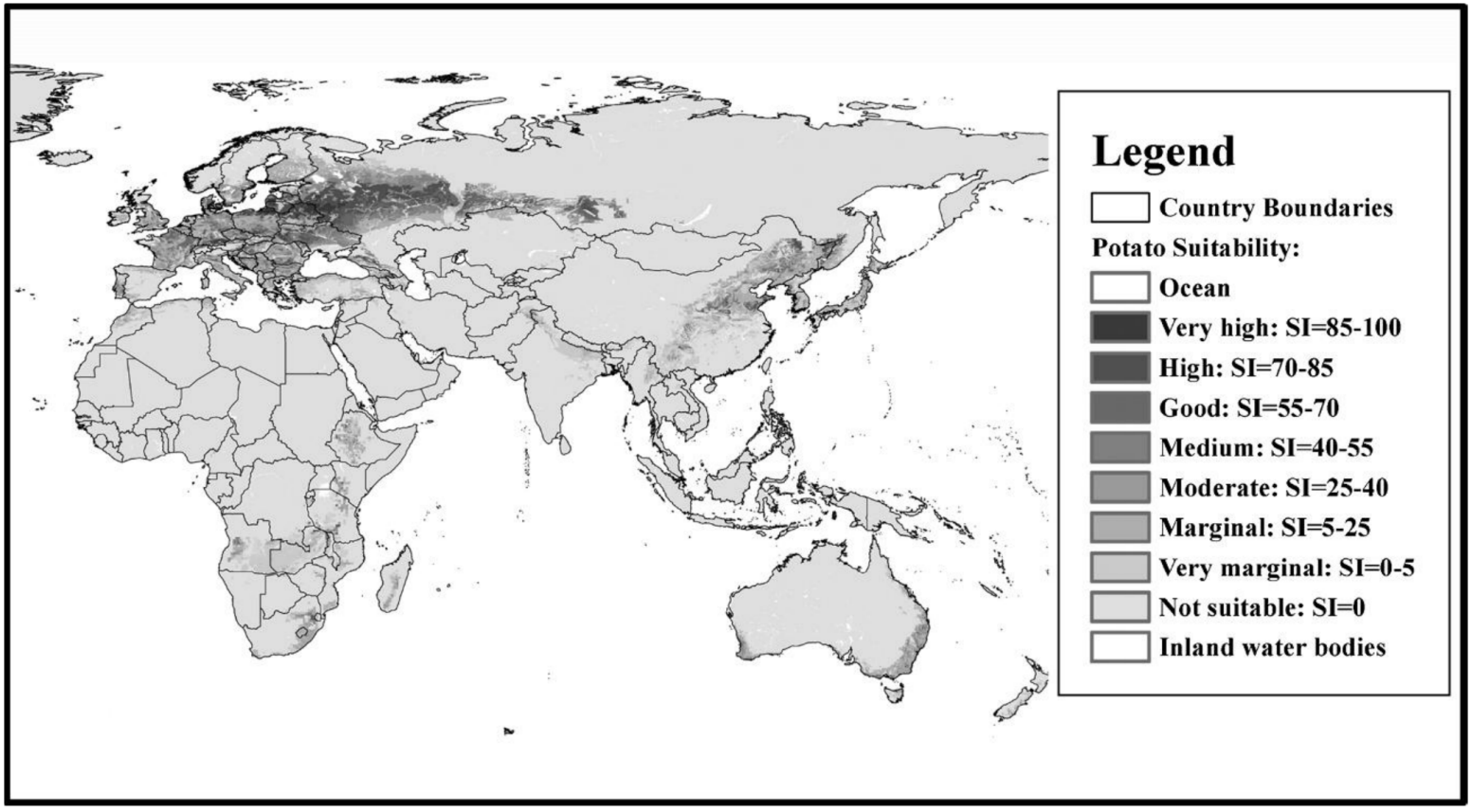


FIGURE II
Average Potato Suitability in the Old World

Nunn and Qian (2011) Figure II, drawn from the same FAO product that is sitting in your `data` folder. Read the legend. It names the nine suitability classes in order, from very high at the dark end through not suitable, and then inland water. It does *not* tell you which integer the file stores for each one. That mapping is an inference, and the next slide checks it.

What one cell holds

```
f <- freq(europe)                                  # europe is the cropped
    layer
cbind(f[, c("value", "count")], label = lbl[f$value])
```

```
##   value count          label
## 1     1    20     very high
## 2     2  3553          high
## 3     3  6643          good
## 4     4 13817        medium
## 5     5 24968      moderate
## 6     6 40821      marginal
## 7     7  6795 very marginal
## 8     8 22077  not suitable
## 9     9  2050  inland water
```

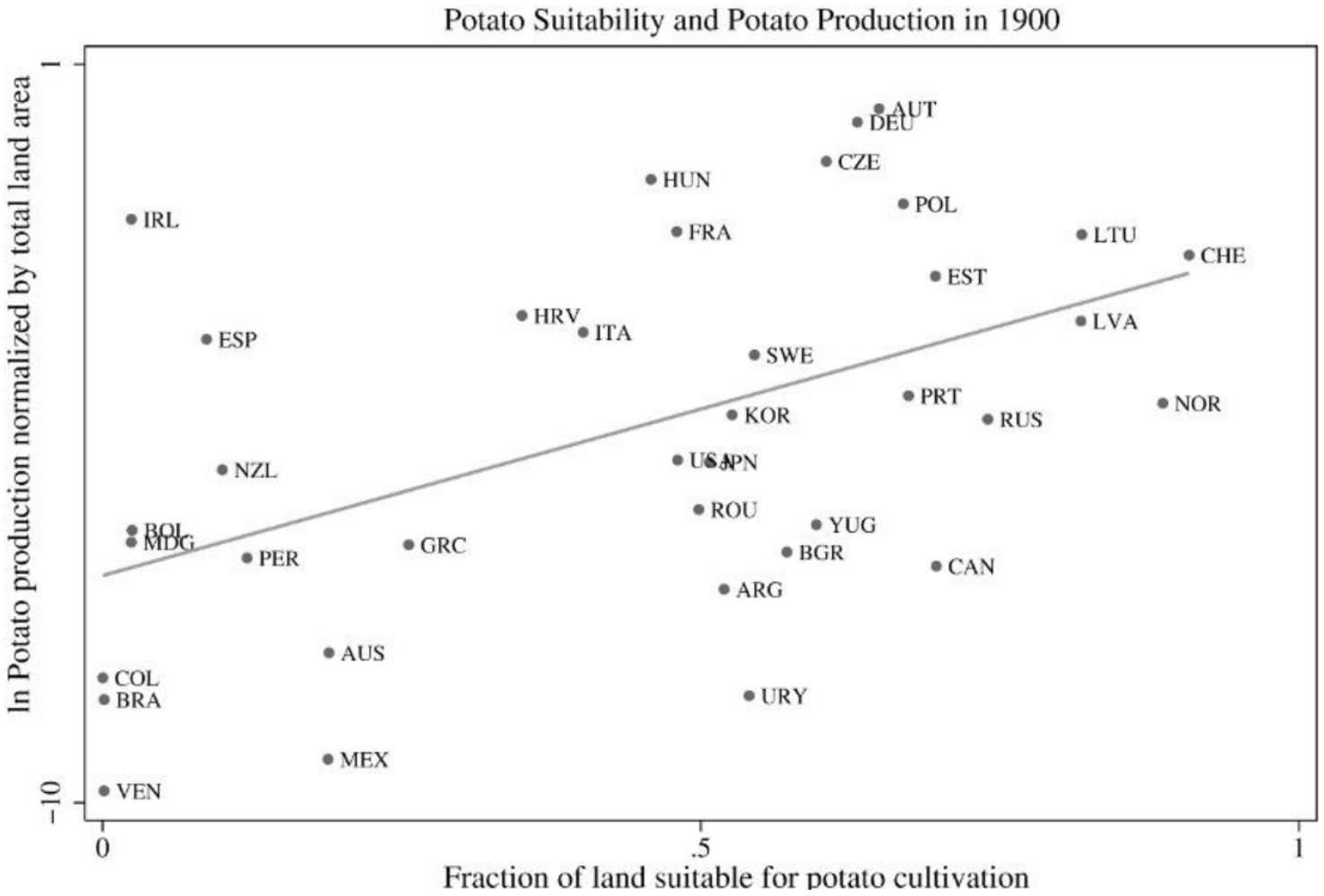
Class 1 is the best land and class 8 is the worst

The scale runs **downward**, and the counts prove it. Class 1 holds 520 cells in the whole world, which is what a top band looks like and is impossible for "not suitable". Nunn and Qian (2011) call land suitable at 40 percent of maximum yield or better, which is classes 1 to 4, and that is 20.2 percent of the land cells in this box. Class 9 is water, and it carries no suitability at all.

The FAO built these from climate, soil, and slope, one crop at a time, assuming rain-fed cultivation at medium input intensity. The file carries none of that. It carries a grid, a CRS, and nine integers.

run it

Does a measure built in the 1990s describe 1700?



Nunn and Qian (2011) Figure III. Their suitability measure against 1900 potato production per unit of country land area, across the 37 countries that report it, in the earliest year with production data. The standardized beta is 0.52. Climate, soil, and slope move slowly, which is the argument, and this is the check that the index tracks what farmers actually did. Week 12 asks the same question of a classifier.

How big is a cell?

```
res(potato)                                          # degrees, not meters
```

```
## [1] 0.08333333 0.08333333
```

```
km2 <- cellSize(potato, unit = "km")
terra::extract(km2, cbind(10, c(0, 30, 50, 60)))[, 1]
```

```
## [1] 85.47965 74.30651 55.42696 43.22625
```

A cell at sixty north is half a cell at the equator

85.5 square kilometers at the equator against 43.2 at sixty north, a factor of 1.98. A degree grid is not an equal-area grid. Average a raster over a continent and you have weighted the tropics.

Crop changes the extent, mask changes the values

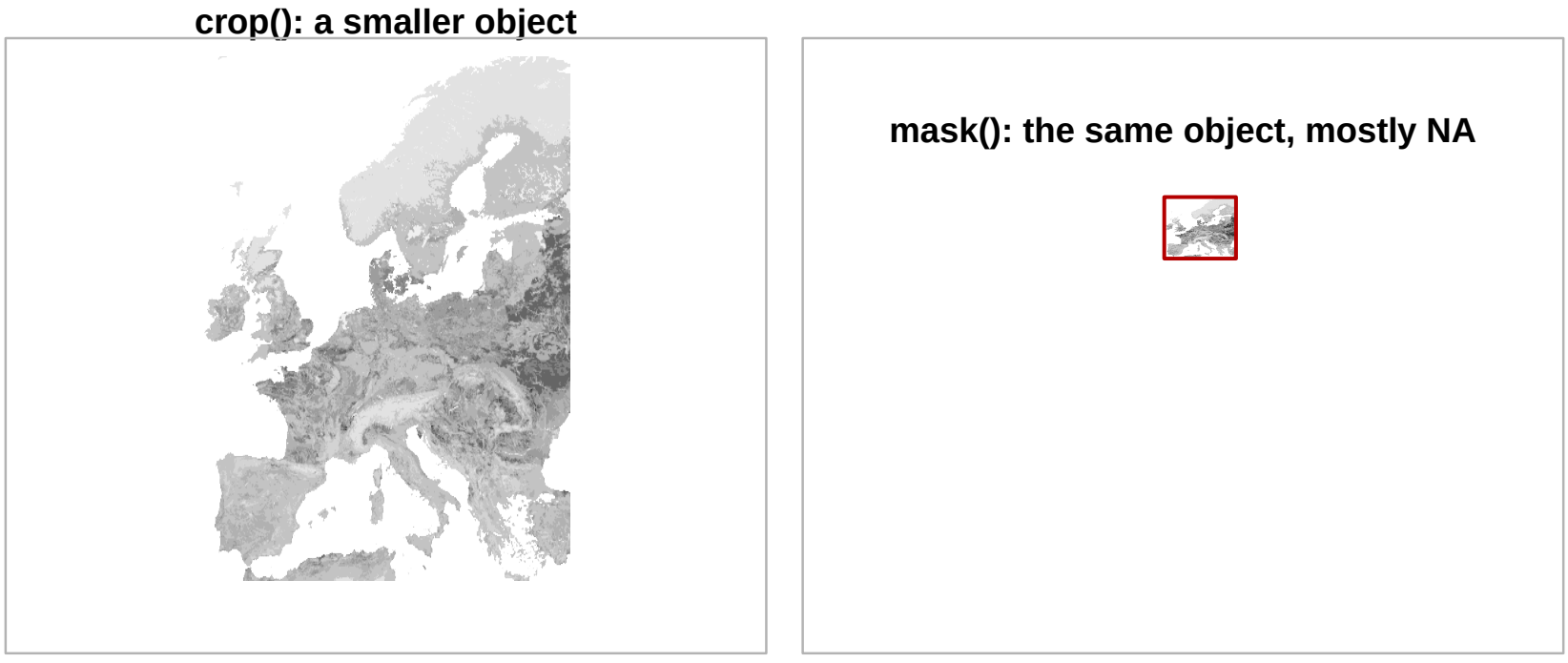
```
cropped <- crop(potato, eu_box)
masked  <- mask(potato, vect(st_as_sfc(st_bbox(
             c(xmin = -11, ymin = 35, xmax = 31, ymax = 71), crs = 4326))
                 ))

c(original = ncell(potato), cropped = ncell(cropped), masked = ncell(
    masked))
```

```
## original  cropped   masked
##  9331200   217728  9331200
```

`crop()` cuts the extent down. `mask()` keeps the extent and sets everything outside to `NA`. Mask when the shape matters and you need the grid to line up with something else. Crop when you want the object smaller.

What crop and mask left behind



Both panels are drawn to the object's own extent. The cropped layer is Europe and holds 217,728 cells. The masked layer is still the whole world and holds 9,331,200, with Europe the only part that survived. Dark is suitable, because low classes are good land.

Map algebra, and reclassifying

Arithmetic on a raster is per cell. This is the local class of map algebra.

```
europe <- crop(potato, eu_box)

good <- europe <= 4                                  # the Nunn and Qian
    threshold
global(good, "mean", na.rm = TRUE)                   # share of cells, water
    included
```

```
##                                 mean
## gaez_potato_suitability 0.1990409
```

```
rcl <- matrix(c(0,4,1, 4,7,2, 7,8,3, 8,9,NA), ncol = 3, byrow = TRUE)
terra::freq(classify(europe, rcl))                   # water became NA and drops
    out
```

```
##   layer value count
## 1     1     1 24033
## 2     1     2 72584
## 3     1     3 22077
```

Local operations are cell by cell. Focal operations use a neighborhood. Zonal operations use a shape. Global operations reduce the whole layer to a number. Four kinds, and tonight uses all four.

A plot is a sample of the data

```
# regular sampling is deterministic, so there is no seed to set
drawn <- spatSample(potato, 5e5, method = "regular", as.raster = TRUE)
c(in_the_layer = ncell(potato), on_the_screen = ncell(drawn))
```

```
##  in_the_layer on_the_screen
##       9331200        500000
```

```
f2  <- merge(as.data.frame(freq(potato)), as.data.frame(freq(drawn)),
             by = "value", suffixes = c("_all", "_drawn"))
pct <- function(x) round(100 * x / sum(x), 2)
head(data.frame(value = f2$value, all = pct(f2$count_all),
                drawn = pct(f2$count_drawn)), 4)
```

```
##   value  all drawn
## 1     1 0.02  0.02
## 2     2 1.20  1.19
## 3     3 2.06  2.04
## 4     4 3.81  3.79
```

The histogram survives. The map does not.

`plot()` draws about 5.4 percent of these cells and every class share survives to within 0.14 of a percentage point. That is the trap. Any summary you compute from the picture is right, so nothing feels wrong, while every feature smaller than the sampling step has silently gone. Pass `maxcell` yourself when detail matters.

predict, then run

Resolution is a choice you are making

```
coarse <- aggregate(europe, fact = 10, fun = "modal", na.rm = TRUE)

rbind(fine   = c(res(europe),  ncell = ncell(europe)),
      coarse = c(res(coarse), ncell = ncell(coarse)))
```

```
##                                  ncell
## fine   0.08333333 0.08333333 217728
## coarse 0.83333333 0.83333333   2244
```

Ten times coarser in each direction means about a hundred times fewer cells and runs about a hundred times faster. Neither 504 nor 432 divides by ten, so `aggregate` quietly pads the extent to make it fit, and `ext(coarse)` is no longer `ext(europe)`. It is also a different variable. The verb matters as much as the factor. `fun = "modal"` keeps the commonest class, which is still a class. `fun = "mean"` would return the average of a hundred category codes, and nothing in the object would remind you that such a thing does not exist.

Focal operations use a neighborhood

A focal operation replaces each cell with a function of the cells around it. You supply the window and the function.

```
w   <- matrix(1, nrow = 5, ncol = 5)          # a 5 by 5 window
dry <- ifel(europe == 9, NA, europe)          # water out BEFORE any
    averaging
sm  <- focal(dry, w = w, fun = mean, na.rm = TRUE)

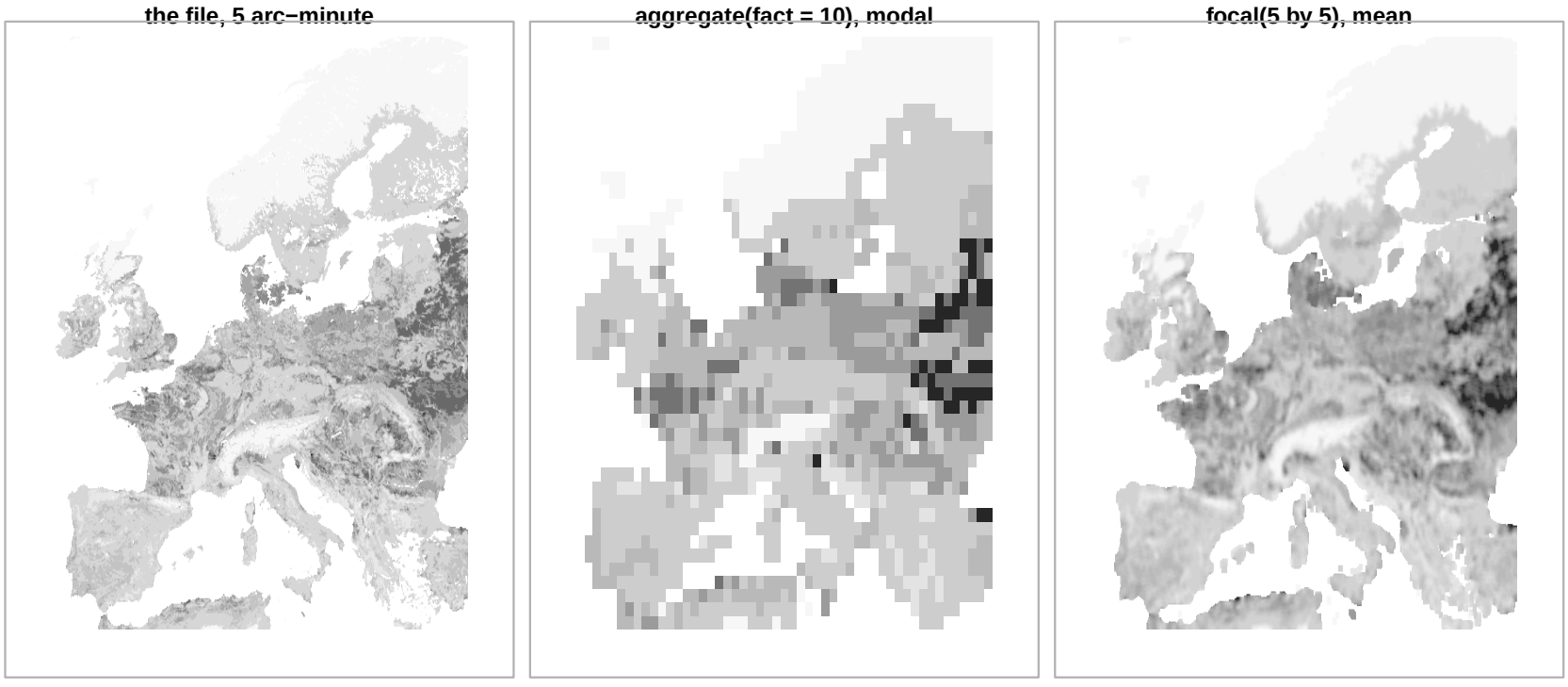
c(original_sd = sd(values(dry), na.rm = TRUE),
  focal_sd    = sd(values(sm),  na.rm = TRUE))
```

```
## original_sd    focal_sd
##    1.540348    1.381731
```

Water has to go before the window runs. A neighborhood mean that includes a 9 returns a number no class ever had, which is the same error as averaging a category. Smoothing then lowers the variance, which is the point and also the danger. This is a defensible way to say that a city farms more than one pixel, and it is a **different treatment variable** from the one under the city. At our 1,017 cities the two correlate 0.695 and differ by 0.51 of a suitability class on average. Focal windows do the real work in Week 7, where a cost surface has to know its neighbors.

run it

Coarsening and smoothing both change the variable



Dark is suitable, and water is dropped from all three. The middle panel still holds whole classes and has lost the Alps. The right panel has kept the shape of everything and holds 1,064 distinct values where the file had 8, so it is no longer a classification of anything. Both are defensible. Neither is the variable you started with.

Never interpolate a category

```
moll <- "+proj=moll +datum=WGS84"        # ESRI:54009, no EPSG code exists
bad  <- project(europe, moll, method = "bilinear")
good <- project(europe, moll, method = "near")

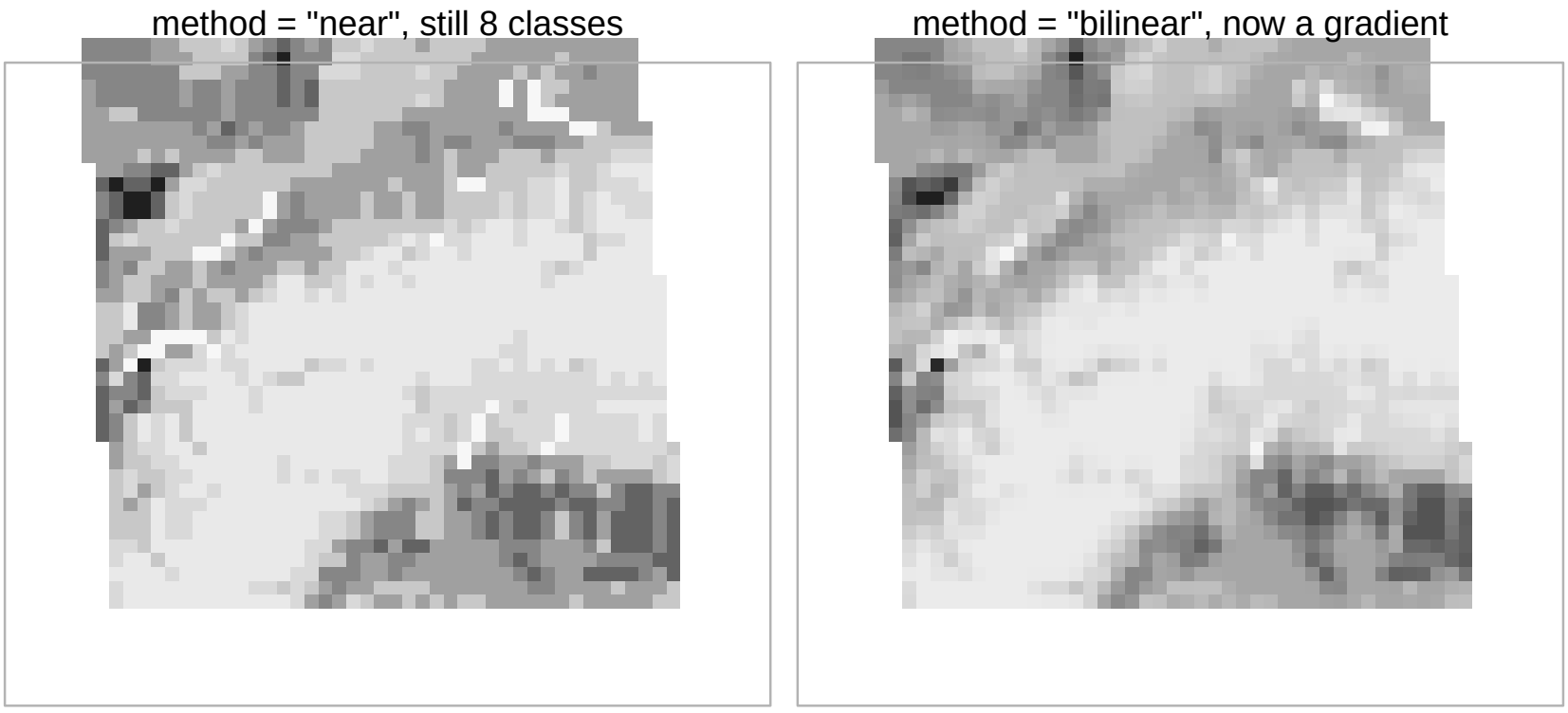
c(classes_in   = length(unique(values(europe, na.rm = TRUE))),
  bilinear_out = length(unique(values(bad,    na.rm = TRUE))),
  nearest_out  = length(unique(values(good,   na.rm = TRUE))))
```

```
##    classes_in bilinear_out  nearest_out
##             9        55062             9
```

Suitability class 8.04 does not exist

Bilinear averages the four nearest cells, which is correct for temperature and meaningless for a category. 53.8 percent of the output is not an integer. Use `method = "near"` for anything categorical, always.

What bilinear did to the Alps



The western Alps and the Po valley, reprojected to Mollweide two ways. On the left the class edges are still edges. On the right they are ramps, and this one corner of the Alps turns 8 classes into 1,102 values. The picture on the right looks better and means less.

Rasters lose something every time you move them

```
there_and_back <- project(project(europe, moll, method = "near"),
                          europe, method = "near")

mean(values(europe) != values(there_and_back), na.rm = TRUE)
```

```
## [1] 0.03929826
```

3.9 percent of cells come back different. Reprojecting a vector moves coordinates and is reversible. Reprojecting a raster **resamples**, and resampling is lossy. Project once, from the original, and never chain.

And the same applies to writing

`writeRaster(x, f, datatype = "INT1U")` on a layer holding halves silently rounds them to integers. Here 9 distinct values become 5. The file is smaller and the data is gone.

Two grids that do not line up

```
rough <- aggregate(europe, fact = 4, fun = "modal", na.rm = TRUE)

rbind(europe = c(res(europe), dim(europe)[1:2]),    # resolution and
      rough  = c(res(rough),  dim(rough)[1:2]))     # dimensions, all four
```

```
##                [,1]       [,2] [,3] [,4]
## europe 0.08333333 0.08333333  432  504
## rough  0.33333333 0.33333333  108  126
```

```
aligned <- resample(rough, europe, method = "near")
both    <- c(europe, aligned)                          # one object, two
    layers
names(both) <- c("fine", "coarse"); nlyr(both)
```

```
## [1] 2
```

Two surfaces can only meet in an expression when they share an origin, a resolution, and a CRS. `resample()` puts one onto the other's grid, and it is the same lossy business as projecting, so pick the method by what the values mean. A multi-layer object is how a stack of surfaces travels together, and it is how Week 8's monthly climate data will arrive.

run it

Rasterize and vectorize

```
grid <- rasterize(vect(pts), europe, fun = "length")    # shapes to
    surface
poly <- st_as_sf(as.polygons(ifel(europe <= 4, 1, NA)))  # surface to
    shapes

c(cities   = nrow(pts),
  cells    = sum(!is.na(values(grid))),
  busiest  = max(values(grid), na.rm = TRUE),
  parts    = nrow(poly))
```

```
##  cities   cells busiest   parts
##    1017    1010       2       1
```

A cell here is about seven kilometers across, so the grid cannot tell Hamburg from Altona four kilometers away. Going the other way, `as.polygons()` dissolves the suitable cells into one shape that `sf` will accept, which is what Week 7 needs when a cost surface has to meet a road network. Sending the unsuitable cells to `NA` first is what keeps the result one shape rather than two.

Break

Ten minutes.

From surface to column

The verb the whole week is about

```
cty$raw  <- terra::extract(europe, vect(pts))[, 2]       # as the file
    ships

suit_r   <- ifel(europe == 9, NA, 9 - europe)            # water out, high
     = good
cty$suit <- terra::extract(suit_r, vect(pts))[, 2]

c(cities = nrow(cty), on_water = sum(cty$raw == 9, na.rm = TRUE),
  usable = sum(!is.na(cty$suit)), median = median(cty$suit, na.rm = TRUE)
       )
```

```
##    cities on_water    usable    median
##      1017        7      1010         3
```

Two lines, 1,017 cities. Recode the **surface** once, before anything touches it, rather than patching the column afterwards. The surface is now a column and everything you know about regression applies again.

Write `terra::extract`, not `extract`

`tidyr` exports an `extract()` too, and whichever package you attached last wins. The error you get talks about arguments while the cause is the namespace, and it will cost you twenty minutes. Name the package.

Seven cities sit on water

```
cty %>% filter(raw == 9) %>%
  select(city_jjk, countryname) %>% print(n = 7)
```

```
## # A tibble: 7 x 2
##   city_jjk   countryname
##   <chr>      <chr>
## 1 AALBORG    Denmark
## 2 KONSTANZ   Germany
## 3 TRONDHEIM  Norway
## 4 EDAM       The Netherlands
## 5 ENKHUIZEN  The Netherlands
## 6 HARDERWIJK The Netherlands
## 7 MEDEMBLIK  The Netherlands
```

Class 9 is the largest number on the scale

Four of these are Zuiderzee ports. Konstanz is on the Bodensee, Aalborg on the Limfjord, and Trondheim on its fjord. The cell underneath each of them is **water**. Take the file at face value, let large mean suitable, and these seven become the most potato-suitable cities in Europe.

A missing value announces itself. A value that is present and plausible and means something else does not. Nothing in `terra` will ever warn you about this one.

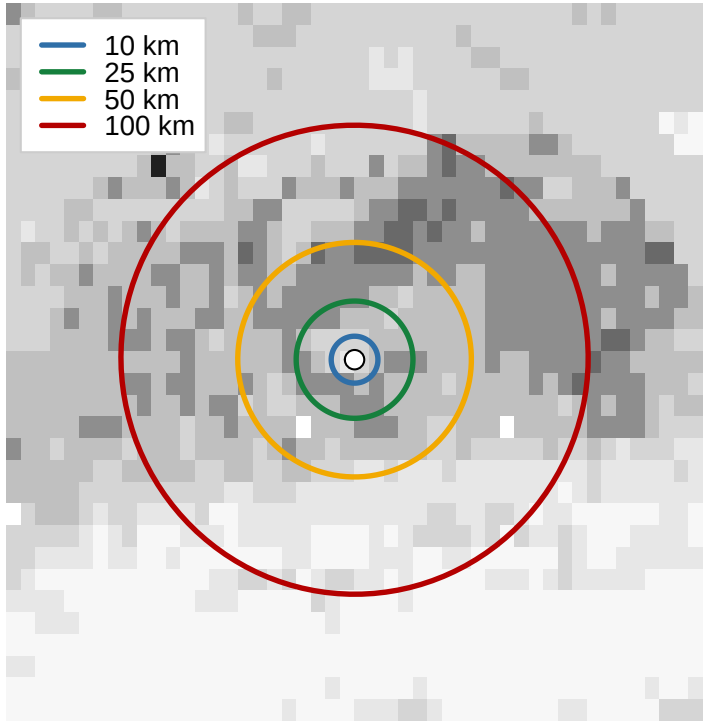
A point, or a neighborhood?

```
for (km in c(10, 25, 50, 100)) {
  buf <- st_buffer(st_transform(pts, 3035), km * 1000) %>% st_transform
       (4326)
  cty[[paste0("suit", km)]] <-                          # exact: these
       buffers are
    terra::extract(suit_r, vect(buf), fun = mean,     # only a few cells
         wide
                   na.rm = TRUE, exact = TRUE)[, 2]
}
sapply(cty[, paste0("suit", c(10, 25, 50, 100))],
       function(v) cor(v, cty$suit, use = "complete.obs"))
```

```
##    suit10    suit25    suit50   suit100
## 0.8417853 0.6795543 0.5746207 0.4664078
```

A city of ten thousand people fed itself from a few kilometers. A city of two hundred thousand did not. You are choosing a measurement, and the choice has **no default**. Nunn and Qian (2011) choose one hundred kilometers, and their footnote 40 reports fifty and two hundred as well. 3035 is ETRS89 Lambert azimuthal equal-area for Europe, used here because `st_buffer` wants meters. It is an equal-area projection doing a distance job, which costs about one percent at the edges of this map, and that is a choice as much as the radius is.

One city and four radii



Munich and its four candidate catchments, on the recoded surface where dark is good land. The hundred-kilometer ring reaches the Alps in one direction and the Danube plain in the other, and those are different farms. The white dot is the single pixel the last slide used. It correlates 0.47 with the hundred-kilometer mean.

Zonal statistics, and the cells on the edge

```
peru <- st_transform(st_read("../data/peru_districts.shp", quiet = TRUE),
      4326)
pr   <- crop(potato, ext(st_bbox(peru)[c(1, 3, 2, 4)]))

plain <- terra::extract(pr, vect(peru), fun = mean, na.rm = TRUE)[, 2]
exact <- terra::extract(pr, vect(peru), fun = mean, na.rm = TRUE, exact =
      TRUE)[, 2]

max(abs(plain - exact), na.rm = TRUE)
```

```
## [1] 1.072915
```

By default a cell counts if its center falls inside. `exact = TRUE` weights each cell by the fraction actually covered. On Peru's districts, which are close in size to a cell, that is worth up to **1.07** of a suitability class. On whole countries it is worth 0.014. The error grows as your polygons **shrink** toward the size of a cell, because it lives on the perimeter and the perimeter is what dominates a small shape.

run it

Building the dose their way

```
suitable <- ifel(potato == 9, NA, potato <= 4)      # classes 1 to 4,
    water out
km2_ok   <- cellSize(potato, unit = "km") * suitable

b100 <- st_buffer(st_transform(pts, 3035), 100000) %>% st_transform(4326)
area <- terra::extract(km2_ok, vect(b100), fun = sum,
                       na.rm = TRUE, exact = TRUE)[, 2]
cty$lnarea <- log(area + 1)                          # the +1 is a choice
    too

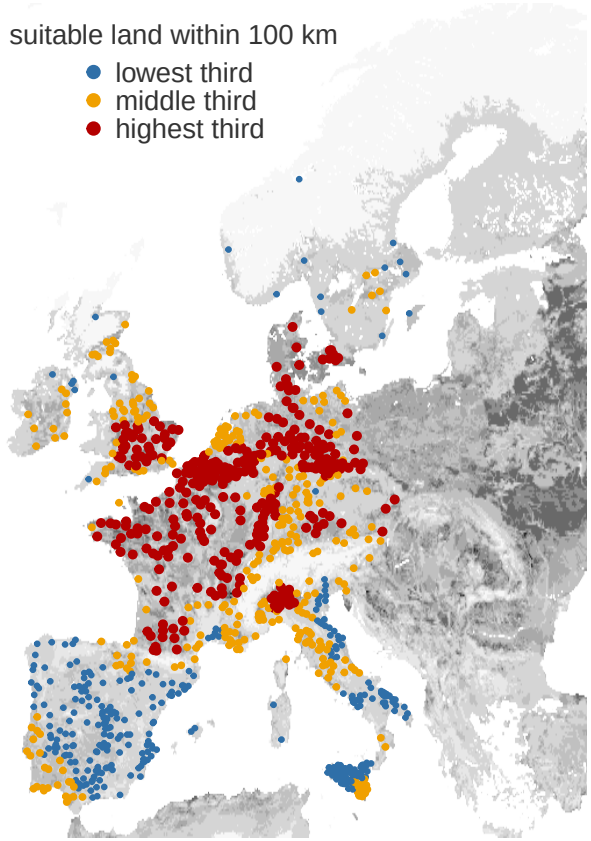
c(cities = sum(!is.na(cty$lnarea)), median_km2 = median(area, na.rm =
    TRUE))
```

```
##     cities median_km2
##   1017.000   2299.442
```

Nunn and Qian (2011) do not use the cell under the city. Their dose is the log of **suitable land area** within a hundred kilometers, where suitable means classes 1 to 4. A threshold, a buffer, a sum and a log. Every one of those is a verb from the last hour, and the buffer is the neighborhood question answered at a hundred kilometers, with `exact = TRUE` because the previous slide said so. This approximates their construction. They sum qualifying land shares inside each cell from the FAO's five categories, while we hold one class per cell. The two line up at the 40 cut.

run it

The dose on the map



Grey is the recoded surface, dark for good land. The dots are the 1,017 cities colored by thirds of suitable land within a hundred kilometers, which is the treatment. The dot and the grey beneath it agree only loosely, at a correlation of 0.34 with the pixel class. A city on poor ground with good ground around it is a high-dose city, and that is what the buffer is for.

run it

The design

Writing it down

For city i in year t, with S_i the log of suitable land within a hundred kilometers of the city and Post_t an indicator for 1700 and later,

$$\ln \mathrm{pop}_{it} = \beta \left(S_i \times \mathrm{Post}_t \right) + \mu_i + \delta_t + \varepsilon_{it}$$

- μ_i eats every fixed difference between cities, including the level of suitability itself, which never changes.
- δ_t eats everything common to a year, including the general rise in European population.

- β is identified by whether high-dose cities **changed trajectory** after 1700 relative to low-dose ones. Standard errors clustered by city.

Build the panel

```
panel <- cty %>%
  select(city_jjk, dose = lnarea, pop1500, pop1600, pop1700, pop1800) %>%
  pivot_longer(starts_with("pop"), names_to = "year", values_to = "pop",
               names_prefix = "pop", names_transform = as.integer) %>%
  filter(pop > 0, !is.na(dose)) %>%
  mutate(lpop = log(pop), post = as.integer(year >= 1700))

c(rows = nrow(panel), cities = n_distinct(panel$city_jjk))
```

```
##   rows cities
##   3746   1016
```

```
panel %>% count(year) %>% pull(n)            # cities observed in each
    year
```

```
## [1]  748  991  994 1013
```

Four population columns become one column and a year. This is `pivot_longer` from Week 2 doing the job it exists for, and a panel is the only shape a difference in differences can read. The panel is unbalanced. A city enters in the first year it has a recorded population, so there are 748 in 1500 and 1,013 in 1800. Entry is an outcome, and the years a city is missing are not missing at random. Nunn and Qian (2011) carry the same problem.

run it

The estimate

```
m <- lm(lpop ~ dose:post + factor(city_jjk) + factor(year), data = panel)

round(coeftest(m, vcov. = vcovCL(m, cluster = panel$city_jjk))["dose:post
    ", ], 4)
```

```
##    Estimate Std. Error    t value   Pr(>|t|)
##      0.0211     0.0079     2.6645     0.0078
```

A one percent increase in suitable land within a hundred kilometers is worth about **0.021 percent higher** city population after 1700, with $t = 2.66$. The sign is theirs. The next slide puts the size beside theirs. Read the standard error with one eye closed.

Clustering by city assumes cities are independent of one another, and suitability is one of the most spatially autocorrelated variables you will ever hold. Week 9 is about what that does to t.

run it

Their city-level estimates

Nunn and Qian (2011) run this design on cities in their Table VIII. The comparable columns are the European ones.

	estimate	clustered s.e.	Conley s.e.
Table VIII (6), Europe, towns of 1,000 or more	0.042	(0.025)	[0.023]
Table VIII (7), the same with country-year effects	0.029	(0.015)	[0.019]
ours, 1,016 cities, four periods	0.021	(0.008)	–

- Same sign, same order of magnitude, from a shapefile, a GeoTIFF and a dozen lines of R. Their controls are ours plus suitability for wheat and rice, log elevation, log ruggedness and log tropical area, each interacted with every year effect, and they run ten periods from 1000 to 1850 where we run four.

- The bracketed column holds **Conley** standard errors, which let nearby cities be correlated and assume independence beyond five degrees. Week 9 is about whether that repair works.

The test the design owes you

```
pre <- panel %>% filter(year < 1700) %>%
  mutate(fake_post = as.integer(year >= 1600))

mp <- lm(lpop ~ dose:fake_post + factor(city_jjk) + factor(year), data =
    pre)
round(coeftest(mp, vcov. = vcovCL(mp, cluster = pre$city_jjk))["dose:fake
    _post", ], 4)
```

```
##    Estimate Std. Error    t value   Pr(>|t|)
##     -0.0123     0.0105    -1.1640     0.2448
```

-0.0123, $t = -1.16$, and the potato has not arrived yet

High-dose and low-dose cities were on the same path from 1500 to 1600. That is the assumption the design rests on, and on this one test it holds. Nunn and Qian (2011) run the same check across six pre-periods and it holds for them too.

Three radii

```
for (km in c(50, 200)) {
  b <- st_buffer(st_transform(pts, 3035), km * 1000) %>% st_transform
       (4326)
  cty[[paste0("lna", km)]] <- log(terra::extract(km2_ok, vect(b), fun =
       sum,
                                      na.rm = TRUE, exact = TRUE)[, 2] + 1)
}
round(rbind(`50 km`  = c(did_at("lna50"),  placebo_t = pl_at("lna50")[3])
     ,
            `100 km` = c(did_at("lnarea"), placebo_t = pl_at("lnarea")
                [3]),
            `200 km` = c(did_at("lna200"), placebo_t = pl_at("lna200")
                [3])), 3)
```

```
##        estimate    se     t cities placebo_t.t
## 50 km     0.016 0.007 2.257   1016      -0.908
## 100 km    0.021 0.008 2.664   1016      -1.164
## 200 km    0.026 0.011 2.314   1016      -1.050
```

The radius was a choice, and their footnote 40 reports that it does not matter. Here it does not matter either. The estimate rises gently with the catchment and every placebo stays quiet. Two diagnostics passed.

Estimate a coefficient for every year

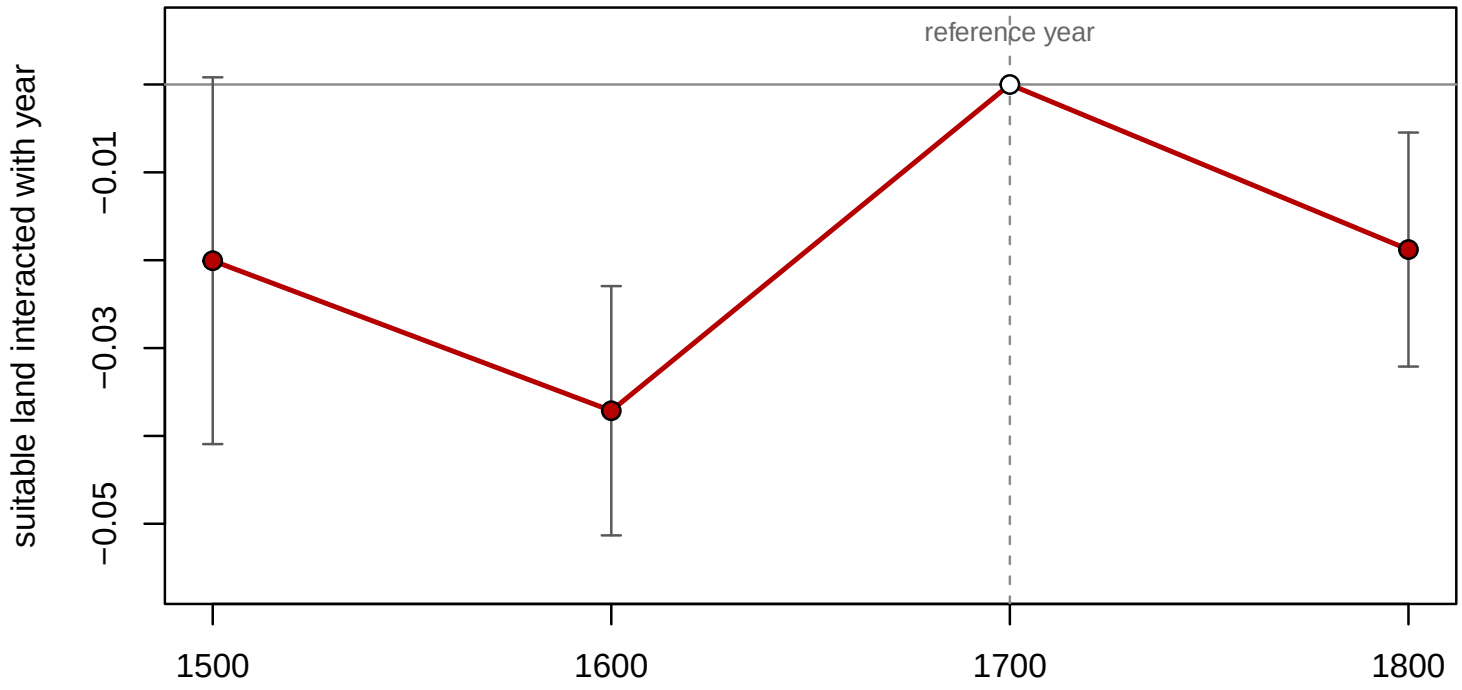
```
yrs <- sort(unique(panel$year))
D   <- sapply(yrs, function(y) panel$dose * (panel$year == y))
colnames(D) <- paste0("d", yrs)
keep <- setdiff(colnames(D), "d1700")        # 1700 is the reference year

es <- lm(reformulate(c(keep, "factor(city_jjk)", "factor(year)"), "lpop")
     ,
         data = cbind(panel, as.data.frame(D[, keep])))
round(coeftest(es, vcov. = vcovCL(es, cluster = panel$city_jjk))[keep, ],
      4)
```

```
##       Estimate Std. Error t value Pr(>|t|)
## d1500  -0.0201     0.0107 -1.8833   0.0598
## d1600  -0.0371     0.0072 -5.1300   0.0000
## d1800  -0.0188     0.0068 -2.7648   0.0057
```

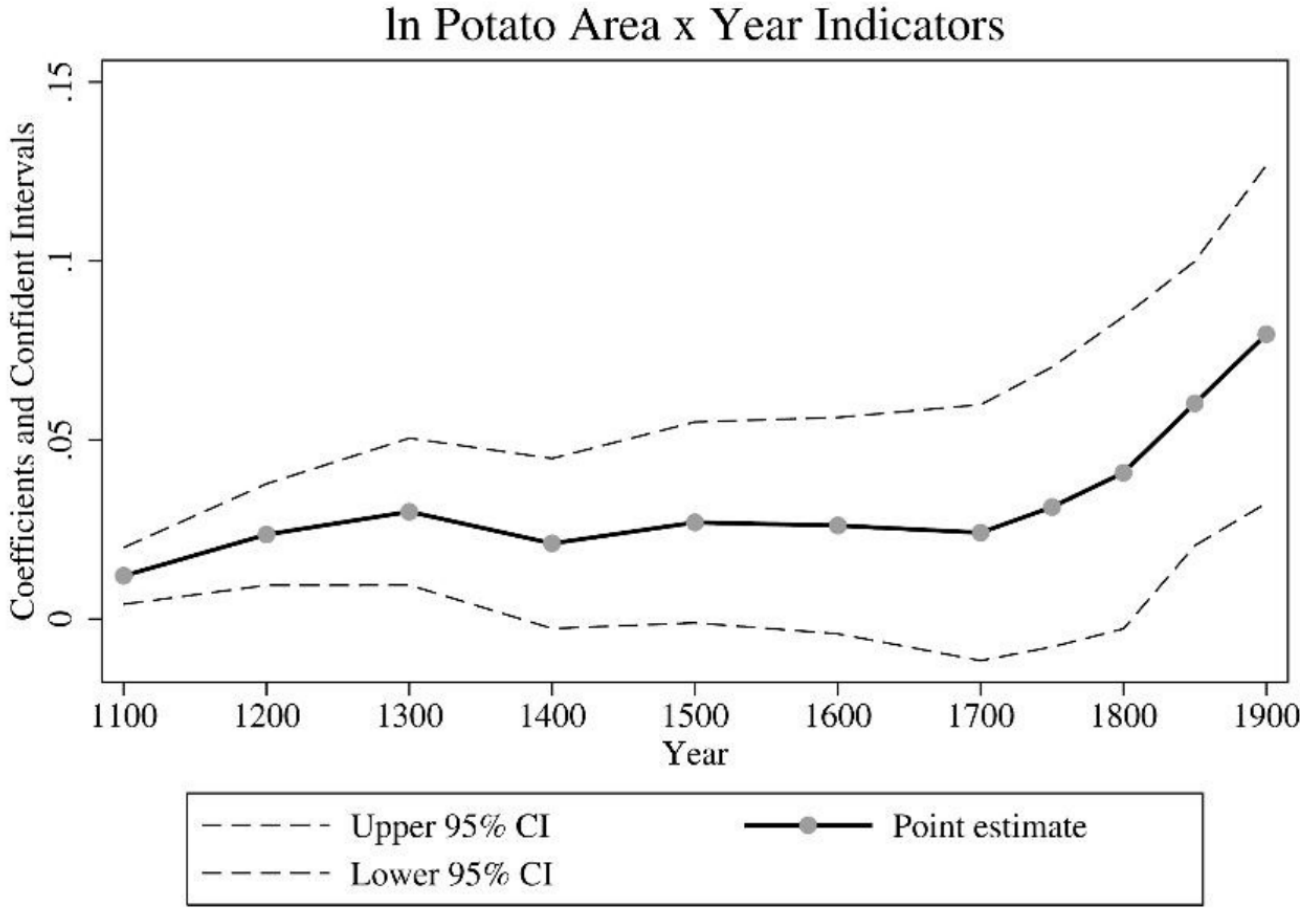
One interaction per year in place of one interaction with post. The reference year has to be dropped, so every coefficient is measured **relative to 1700** and the level of any single one carries no information. The shape does, and this is the test that the placebo could not run, because the placebo only looked backward.

The rise comes a century early



Ninety-five percent intervals, clustered by city. Relative to 1700, high-dose cities were 0.037 lower in 1600 and 0.019 lower in 1800. The only rise in the series is the step from 1600 into 1700, and by their own dating the potato is not a field crop until after it. The positive estimate two slides back is that step. With one period on each side of 1700 this panel cannot tell an early potato from something else entirely, and that is why they need twelve.

Their version of the same picture



(a) ln Total Population

Nunn and Qian (2011) Figure IVa, log total population at the country level, suitability interacted with each period. Their base year is 1000, so the levels are arbitrary in exactly the way ours are, and the shape is the whole claim. Theirs holds a level for six centuries and turns after 1750. Ours has one step, and it lands a century before theirs does.

The date was a choice too

```
round(rbind(`post from 1700, ours`   = did_at("lnarea", cut = 1700),
            `post from 1800, theirs` = did_at("lnarea", cut = 1800)), 4)
```

```
##                          estimate     se      t cities
## post from 1700, ours       0.0211 0.0079 2.6645   1016
## post from 1800, theirs     0.0001 0.0077 0.0157   1016
```

Nunn and Qian (2011) spend several pages defending 1750 as the first post-adoption period, drawing on the historical record of when the potato became a field crop. On their dating, 1700 belongs to the pre period. Our four years are 1500, 1600, 1700 and 1800, so adopting their cutoff leaves one year after it, and the estimate falls to 0.0001 with $t = 0.02$.

The event study already told you this

The whole effect was the step into 1700. Date the treatment after that step and there is nothing left. Two diagnostics passed and the third one, the one that reads the paper's own chronology, did not.

The shortcut

```
round(rbind(`buffer, 100 km`   = c(did_at("lnarea"), placebo_t = pl_at("
    lnarea")[3]),
            `pixel under city` = c(did_at("suit"),   placebo_t = pl_at("
                suit")[3])), 3)
```

```
##                   estimate    se      t cities placebo_t.t
## buffer, 100 km       0.021 0.008  2.664   1016      -1.164
## pixel under city    -0.089 0.024 -3.647   1009      -2.325
```

Wrong sign, and its placebo fires

Replace the buffer with the pixel and the same regression returns -0.089 with $t = -3.65$, which says better potato land grew slower, and its pre-trend placebo fires at $t = -2.33$. Nothing about the potato changed. One column did.

Dublin sits on a cell the FAO rates very marginal, with only 7 of our 1,010 cities on worse ground, while the land within a hundred kilometers of it averages 3.8. A model writes the pixel version instantly, because it is one line. It is not Ireland's potato ground.

So what do you do about it?

- Nunn and Qian (2011) work over twelve periods, control for suitability for other crops and for geography interacted with every year, restrict to the Old World, and defend the timing at length. Their Figure IVa is the picture our four periods cannot draw.

- Two of their diagnostics we could run passed. The third, their own chronology, removed the result. That is a fair summary of what a four-period panel can say about a twelve-period claim.

- The literature has moved on. Callaway et al. (2024) show that two-way fixed effects with a continuous dose recovers a weighted average that is hard to interpret even with two periods, which is exactly the regression we ran, and that it needs **strong parallel trends**, meaning parallel at every dose rather than on average. Roth et al. (2023) survey the rest.

The appendix carries the coastline confound, the reclassify trap, and what a plot throws away, for anyone who asks.

What it cost

What it cost you to build

One line

`terra::extract(europe, vect(pts))`. That is the two-line version of a paper in the *Quarterly Journal of Economics*, and it has the wrong sign.

- A model writes that line instantly, and writes the regression too.
- It will not tell you that nine means water, that low numbers are good land, that Dublin's pixel is very marginal, or that the version which agrees with the paper stops agreeing the moment you use the paper's own dates.
- Three weeks, three designs, three diagnostics that the **two-line version failed**. That is the job.

Tonight in one slide

1. A raster is one geometry and many values. A degree grid is not equal area.

2. Read the codebook before the column. Here low numbers are good land and the largest number is water.

3. `crop` changes the extent, `mask` changes the values. A plot is a sample of the data. Never resample a categorical layer with `bilinear`.

4. `terra::extract` is one line and at least four decisions. Zonal error is worst for the smallest polygons, so use `exact = TRUE` when a shape is only a few cells wide.

5. A difference in differences owes you its pre-trends, and an event study shows them. Ours matched their sign and size, passed two checks, and lost the result to their own chronology.

Before next week

Install QGIS before Monday

Week 5 is the making-data week and we work in QGIS the whole evening. Download it, open it once, and confirm it launches. Do not leave this to Sunday night.

1. Read Nunn and Qian (2011), the pre-trend discussion twice, and Table VIII once.
2. Work tonight's script by hand. Rebuild the dose at a radius nobody used tonight.
3. Bring a surface from your own field. We will ask what one line of `extract` would do to it, and what the largest value in it means.

Reminder. The Week 6 lab exam is closed book and closed model, and it is two weeks away.

References I

Callaway, B., Goodman-Bacon, A., and Sant'Anna, P. H. C. (2024). Difference-in-differences with a continuous treatment. NBER Working Paper 32117, National Bureau of Economic Research.

Nunn, N. and Qian, N. (2011). The potato's contribution to population and urbanization: Evidence from a historical experiment. *The Quarterly Journal of Economics*, 126(2):593–650.

Roth, J., Sant'Anna, P. H. C., Bilinski, A., and Poe, J. (2023). What's trending in difference-in-differences? a synthesis of the recent econometrics literature. *Journal of Econometrics*, 235(2):2218–2244.

Appendix

Reading a reclassify matrix

Three columns and one row per interval. `from`, `to`, `becomes`, filled across by `byrow`.

```
probe <- rast(nrows = 1, ncols = 9)          # nine cells, one per class
values(probe) <- 1:9
cl <- function(v) values(classify(probe, matrix(v, ncol = 3,
                                                byrow = TRUE)))[, 1]

rbind(class   = 1:9,
      from_0  = cl(c(0,4,1,  4,7,2,  7,8,3,  8,9,NA)),
      from_1  = cl(c(1,4,1,  4,7,2,  7,8,3,  8,9,NA)),
      relabel = cl(c(1,4,10, 4,7,20, 7,8,30, 8,9,NA)))
```

```
##         [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## class      1    2    3    4    5    6    7    8    9
## from_0     1    1    1    1    2    2    2    3   NA
## from_1     1    1    1    1    2    2    2    3   NA
## relabel    1   10   10   10   20   20   20   30   NA
```

`from_1` is wrong and prints the right answer anyway

Every interval is `(from, to]`, open on the left, so `(1,4]` misses class 1. No other row covers it, and `classify` leaves an uncovered value **unchanged** rather than sending it to `NA`. Class 1 comes back as 1, which is also the code the first bin uses. Relabel the bins and the pass-through appears.

That is 20 cells here and 520 worldwide riding on a leading zero that looks like a typo. `right = FALSE` flips every interval to `[from, to)`, which moves class 4 into the second bin and class 8 into `NA`.

What the sample throws away

```
eu <- crop(potato, eu_box); ed <- crop(drawn, eu_box)
lake <- function(r) patches(ifel(r == 9, 1, NA), directions = 8)

round(c(step       = res(drawn)[1] / res(potato)[1],
        pct_full   = 100 * mean(values(eu) == 9, na.rm = TRUE),
        pct_drawn  = 100 * mean(values(ed) == 9, na.rm = TRUE),
        n_full     = length(unique(values(lake(eu), na.rm = TRUE))),
        n_drawn    = length(unique(values(lake(ed), na.rm = TRUE)))), 2)
```

```
##       step   pct_full  pct_drawn     n_full    n_drawn
##       4.32       1.70       1.83     412.00      63.00
```

The share of water survives and the lakes do not

`plot()` keeps one cell in 19, a step of 4.3 cells each way, about 26 kilometers at fifty north. A regular sample is a fair draw from the marginal distribution, so the share of water returns almost exactly. Nothing about it can hold a feature narrower than its own step. Europe carries 412 separate bodies of inland water at full resolution and 63 in the picture.

Of the 7 cities in tonight's sample standing on a water cell, 3 read a land class once the layer is drawn. Enkhuizen, Harderwijk and Medemblik lose the Zuiderzee and come out on farmland. We meet those cities after the break, and nothing on this map would have told you.

run it

A dose that should do nothing

```
land    <- ifel(is.na(potato) | potato == 9, NA, 1)
all_km2 <- terra::extract(cellSize(potato, unit = "km") * land,
                          vect(b100), fun = sum, na.rm = TRUE, exact =
                              TRUE)[, 2]
cty$lnland <- log(all_km2 + 1)

round(rbind(`ln ALL land within 100 km` = did_at("lnland")), 4)
```

```
##                             estimate     se      t cities
## ln ALL land within 100 km  -0.3616 0.0825 -4.382   1016
```

$t = -4.38$ on a variable with no potatoes in it

A city with more land around it grew more slowly after 1700, which is mostly a statement about ports. It correlates 0.39 with the suitable-area dose, so leaving it out was never harmless. Put both in the same regression and the clean result from their variable moves from 0.0211 to 0.0448.

Work out the direction before you guess it. The omitted dose enters negatively and correlates positively with the one we kept, so dropping it pushed the estimate down. A confound does not always flatter you, and which way it pushes is arithmetic rather than intuition.